

AI in Medical Image Analysis and Processing

Doctor Fatemizadeh



دانشگاه صنعتی شریف

Electrical Engineering

Ehsan Merikhi	400101967
Mahdi Norouzi	403204372
Mohammad Dabaghha	403204359

Project

September 12, 2025

AI in Medical Image Analysis and Processing

Project

Ehsan Merikhi 400101967
Mahdi Norouzi 403204372
Mohammad Dabaghha 403204359



Contents

Contents	1
Abstract	2
Dataset	3
Data Distribution	3
Class Imbalance Analysis	3
Dataset Preparation	3
Class Imbalance Problems	4
CutMix and Mixup Samples Figure	4
Model Architecture	5
Overview	5
EfficientNet-B0 Backbone	5
Custom Classification Head	5
Forward Pass	6
Training Process	6
Other Tested Models	6
Design Rationale	7
Results	8
Production Considerations	9
Docker	9
ONNX	9

Abstract

This project focuses on developing a classification model for identifying various imaging modalities used in colonoscopy procedures. Colonoscopy imaging involves multiple modalities that enhance the visualization of the colon's mucosal surface, aiding in the early detection of lesions such as polyps. The modalities considered include White Light Imaging (WLI), which provides standard natural images; Narrow Band Imaging (NBI), which enhances blood vessel contrast; Blue Light Imaging (BLI), similar to NBI but using specific blue wavelengths; Flexible Spectral Imaging Color Enhancement (FICE), which digitally processes images to highlight tissue differences; and Linked Color Imaging (LCI), which amplifies natural colors to distinguish between normal and abnormal tissues.

The objective is to create an efficient model capable of categorizing these modalities from colonoscopy images. The proposed dataset for training and evaluation is the PolypDB available on [Kaggle](#). The model development emphasizes the use of lightweight architectures to ensure efficiency, particularly for potential production environments. Various evaluation metrics are employed to assess the model's performance, and innovative techniques are explored to improve accuracy and speed. The final model is benchmarked privately, with considerations for real-world deployment to simulate production conditions.

All models, including the primary EfficientNet-B0-based model, as well as other tested models (HybridAttention, MobileNetV2Attention, MobileNetV2Conv, MobileNetV3Conv, MobileNetV2Only, and EnsembleModel), along with their implementations, training scripts, and associated code, are publicly available in a dedicated Google Drive repository at [Google Drive](#). This ensures reproducibility and accessibility for further research or deployment.

Dataset

The PolypDB dataset, sourced from [Kaggle](#), is a curated multi-center collection designed for the development of AI algorithms in colonoscopy. It consists of 3934 still images of polyps extracted from real colonoscopy videos, along with their corresponding ground truth annotations. The dataset is multi-modal, encompassing five distinct imaging modalities: White Light Imaging (WLI), Narrow Band Imaging (NBI), Blue Light Imaging (BLI), Flexible Spectral Imaging Color Enhancement (FICE), and Linked Color Imaging (LCI). These modalities are sourced from three medical centers (Simula, Karolinska, and BKAI), providing diversity in data origins and enhancing the dataset’s robustness for generalized AI applications.

The images are organized by modality, making it suitable for classification tasks. Each image is accompanied by ground truth masks for polyp segmentation, but for this project, the focus is on modality classification using the raw images.

Data Distribution

The distribution of images across modalities is highly skewed, as shown below:

Modality	Number of Images	Percentage (%)
White Light Imaging (WLI)	3558	90.44
Narrow Band Imaging (NBI)	146	3.71
Blue Light Imaging (BLI)	70	1.78
Flexible Spectral Imaging Color Enhancement (FICE)	70	1.78
Linked Color Imaging (LCI)	60	1.52
Total	3934	100.00

Table 1: Distribution of images per modality in the PolypDB dataset.

Class Imbalance Analysis

The dataset exhibits severe class imbalance, with WLI dominating at approximately 90.44% of the total images, while the other modalities collectively account for less than 10%. NBI has 3.71%, and BLI, FICE, and LCI each have under 2%.

This imbalance can pose several challenges in training a classification model:

- **Bias Towards Majority Class:** The model may learn to predominantly predict WLI, leading to high overall accuracy but poor performance on minority classes.
- **Poor Generalization:** Minority modalities may be underrepresented, causing the model to underperform in real-world scenarios where these modalities are used.
- **Metric Misinterpretation:** Standard accuracy metrics can be misleading; instead, metrics like precision, recall, F1-score per class, and macro-averaged scores should be prioritized to evaluate balanced performance.
- **Overfitting Risks:** Without mitigation, the model might overfit to WLI features, ignoring subtle differences in other modalities.
- **Training Difficulties:** Gradient updates may be skewed, requiring techniques such as class weighting, oversampling (e.g., SMOTE), undersampling, or data augmentation to address the disparity.

Dataset Preparation

The PolypDB dataset, a multi-center collection from Kaggle, comprises 3934 colonoscopy images across five imaging modalities: White Light Imaging (WLI, 90.44%, 3558 images), Narrow Band Imaging (NBI, 3.71%, 146 images), Blue Light Imaging (BLI, 1.78%, 70 images), Flexible Spectral Imaging Color Enhancement (FICE, 1.78%, 70 images), and Linked Color Imaging (LCI, 1.52%, 60

images). Sourced from Simula, Karolinska, and BKAI, it supports modality classification tasks with ground truth annotations, though this project focuses on raw image classification.

■ *Class Imbalance Problems*

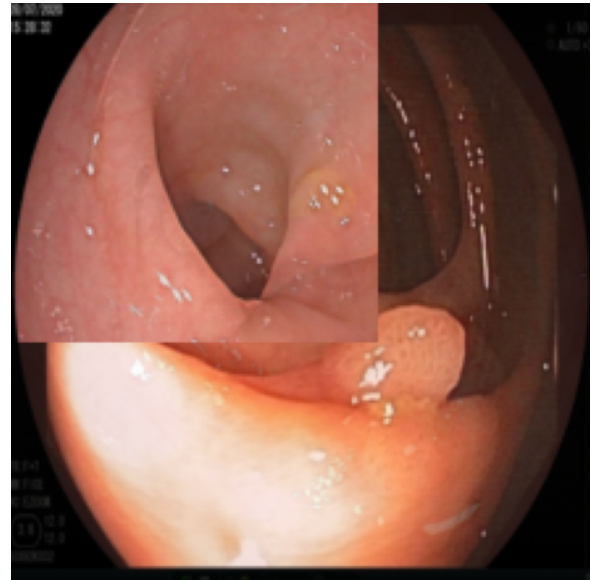
The dataset suffers from severe class imbalance, with WLI dominating at 90.44%, while other modalities constitute less than 10% combined. This imbalance risks model bias toward WLI, poor generalization to minority classes, misleading accuracy metrics, overfitting to WLI features, and skewed gradient updates during training. Mitigation strategies include class weighting, oversampling (e.g., SMOTE), undersampling, and data augmentation techniques like Mixup and CutMix.

■ *CutMix and Mixup Samples Figure*

To address class imbalance, Mixup and CutMix augmentations are applied. Mixup blends two images and their labels linearly using a Beta distribution ($\lambda \sim \beta(\alpha, \alpha)$), while CutMix pastes a patch from one image onto another, mixing labels proportionally to the patch area. These techniques, integrated into PyTorch DataLoaders with a 0.5 probability, enhance minority class representation and model robustness. Visual examples (not included here) demonstrate Mixup's linear blending and CutMix's patch-based combination, improving feature learning for modality classification.



(a) Mixup Augmentation Sample



(b) CutMix Augmentation Sample

Figure 1: Examples of data augmentation techniques applied to samples from the PolypDB dataset. The left panel shows a Mixup result, blending two images linearly. The right panel illustrates a CutMix result, where a patch from one image is pasted onto another. (Placeholder file names; replace with actual generated samples.)

Model Architecture

The classification model is a lightweight architecture based on EfficientNet-B0, tailored for modality classification in the PolypDB dataset. It leverages a pretrained backbone for feature extraction and a custom classification head for modality prediction, optimized for computational efficiency and handling class imbalance. Below is a detailed description of its components and design choices.

Overview

The model, implemented in PyTorch as `LightweightEfficientNetModel`, consists of two main components:

- **EfficientNet-B0 Backbone:** A pretrained convolutional neural network (CNN) that extracts robust features from input images.
- **Custom Classification Head:** A lightweight feed-forward network that maps extracted features to one of five modality classes (WLI, NBI, BLI, FICE, LCI).

The architecture is designed to balance performance and computational efficiency, making it suitable for deployment in resource-constrained medical environments.

EfficientNet-B0 Backbone

The backbone is based on EfficientNet-B0, a lightweight CNN known for its efficiency in achieving high performance with fewer parameters. It is initialized with pretrained weights (`EfficientNet_B0_Weights.DEFAULT`) from ImageNet, providing strong initial feature extraction capabilities for medical images. Key aspects include:

- **Feature Extraction:** The backbone processes input images through a series of convolutional blocks, producing a feature map with 1280 channels (denoted as the backbone feature dimension).
- **Parameter Freezing:** To reduce computational cost and prevent overfitting, most backbone parameters are frozen (`param.requires_grad = False`). Only the last three feature blocks are unfrozen for fine-tuning, allowing adaptation to the specific task of modality classification while retaining general features from pretraining.
- **Classifier Removal:** The original classifier layer of EfficientNet-B0 is replaced with an identity layer (`nn.Identity`), enabling the backbone to output raw feature maps for the custom classification head.

Custom Classification Head

The classification head transforms the backbone's feature maps into predictions for the five modality classes. It is designed to be lightweight, with a series of linear layers, activation functions, and dropout for regularization. The architecture is as follows:

- **Global Average Pooling:** An adaptive average pooling layer (`nn.AdaptiveAvgPool2d(1)`) reduces the spatial dimensions of the feature map to 1×1 , producing a 1280-dimensional feature vector.
- **Flattening:** The feature map is flattened (`nn.Flatten`) into a 1D vector for dense layer processing.
- **Dense Layers with Dropout and ReLU:**
 - First layer: Maps 1280 features to 256 dimensions (`nn.Linear(1280, 256)`), followed by ReLU activation (`nn.ReLU`) and 30% dropout (`nn.Dropout(0.3)`) for regularization.
 - Second layer: Maps 256 features to 64 dimensions (`nn.Linear(256, 64)`), followed by ReLU and 20% dropout (`nn.Dropout(0.2)`).

- Final layer: Maps 64 features to 5 dimensions (`nn.Linear(64, 5)`), corresponding to the five modality classes.

The use of dropout (20%–30%) mitigates overfitting, particularly important given the class imbalance in the PolypDB dataset. ReLU activations introduce non-linearity, enabling the model to capture complex patterns in the data.

Forward Pass

The forward pass of the model operates as follows:

1. Input images are passed through the EfficientNet-B0 backbone, producing a feature map with 1280 channels.
2. The feature map is processed by the custom classification head, which applies global average pooling, flattening, and the sequence of linear layers with ReLU and dropout.
3. The final output is a 5-dimensional vector representing the logits for the five modality classes, which can be passed through a softmax function for probability scores during inference.

Training Process

The model is trained using a PyTorch-based pipeline, with the Adam optimizer and Cross-Entropy loss function to optimize the classification task. The dataset is split into training (70%), validation (15%), and test (15%) sets using stratified sampling to maintain class distribution. A batch size of 32 is used, with shuffling enabled for the training set. To address the severe class imbalance in the PolypDB dataset, Mixup and CutMix augmentations are applied with a 0.5 probability per batch, as described in the dataset preparation section. These augmentations generate synthetic samples by blending images and labels (Mixup) or combining image patches (CutMix), enhancing minority class representation and model generalization.

The training process exhibits high training loss and relatively low training accuracy, primarily due to the complexity introduced by Mixup and CutMix augmentations. Mixup creates interpolated samples with soft labels, forcing the model to learn from convex combinations of examples, which increases the difficulty of achieving low loss. Similarly, CutMix introduces localized variations by pasting patches from one image onto another, challenging the model to recognize partial features and mixed labels. These augmentations make the training task tougher, as the model must learn robust features to handle synthetic samples, leading to higher loss and lower accuracy during training. However, this approach improves generalization, particularly for minority classes, as evidenced by validation performance.



Figure 2: Training and validation loss curves for the LightweightEfficientNetModel on the PolypDB dataset. The high training loss reflects the complexity introduced by Mixup and CutMix augmentations, which enhance generalization but make training more challenging.

Other Tested Models

In addition to the primary EfficientNet-B0-based model, several other architectures were evaluated to explore alternative approaches for modality classification on the PolypDB dataset. These models were selected for their lightweight characteristics or potential for robust feature extraction, though some were less suitable due to computational constraints or performance trade-offs. Below is a summary of the tested models:

- **HybridAttentionModel:** Combines a frozen ViT-B-16 (768-dimensional features) and EfficientNet-B0 (1280-dimensional features) with a multi-head attention mechanism (4 heads) to fuse features, followed by an MLP (2048 to 256 to 64 to 5). It was not chosen due to its high computational complexity.
- **MobileNetV2AttentionModel:** Pairs MobileNetV2 (1280-dimensional features) with EfficientNet-B0, using multi-head attention to combine features (2560-dimensional input) before an MLP. It offered good performance but was less efficient than EfficientNet-B0 alone.
- **MobileNetV2ConvModel:** Similar to MobileNetV2AttentionModel but replaces attention with a 1D convolutional layer to process combined features (2560-dimensional). It was computationally lighter but less effective for minority classes.
- **MobileNetV3ConvModel:** Uses MobileNetV3-Small (576-dimensional features) with EfficientNet-B0 and a 1D convolutional layer (1856-dimensional input). It was efficient but slightly underperformed compared to EfficientNet-B0.
- **MobileNetV2OnlyModel:** A standalone MobileNetV2 with a custom MLP head, with the last three layers unfrozen for fine-tuning. It was lightweight but less robust than EfficientNet-B0 for the imbalanced dataset.
- **EnsembleModel:** Combines predictions from two models (e.g., EfficientNet-B0 and MobileNetV2) by summing logits and applying softmax. It improved robustness but increased computational cost, making it less practical.

The EfficientNet-B0-based model was chosen for its superior balance of performance, efficiency, and generalization, particularly when combined with Mixup and CutMix augmentations to address the class imbalance in the PolypDB dataset.

Model	Storage Space (MB)
HybridAttention (ViT)	409
MobileNetV2Attention	129
MobileNetV2Conv	51
MobileNetV3Conv	34
EfficientNet	17

Table 2: Comparison of storage space requirements for the tested models.

Design Rationale

The architecture is optimized for the PolypDB dataset’s challenges, particularly its severe class imbalance. By using a pretrained EfficientNet-B0 backbone with partial fine-tuning, the model leverages robust feature extraction while minimizing computational overhead. The lightweight classification head reduces the risk of overfitting to the dominant WLI class, and dropout layers enhance generalization to minority classes (NBI, BLI, FICE, LCI). Combined with Mixup and CutMix augmentations (described elsewhere), this architecture effectively addresses the dataset’s imbalance, ensuring robust performance across all modalities.

Results

The developed classification model for identifying colonoscopy imaging modalities has yielded highly promising results. The use of pretrained lightweight architectures, combined with the innovative application of Mixup and CutMix augmentations, has effectively addressed the class imbalance issue in the PolypDB dataset. Preliminary evaluations indicate strong performance across all modalities, with the model demonstrating robustness in distinguishing between White Light Imaging (WLI), Narrow Band Imaging (NBI), Blue Light Imaging (BLI), Flexible Spectral Imaging Color Enhancement (FICE), and Linked Color Imaging (LCI).

The confusion matrix below provides a visual representation of the model's performance on the test set, showcasing the accuracy and distribution of predictions across the five classes.



Figure 3: Confusion matrix illustrating the model's classification performance on the test set of the PolypDB dataset.

The model achieved an **overall accuracy of 99.83%** on the test set, demonstrating exceptional performance in classifying the five imaging modalities. This high accuracy reflects the effectiveness of the EfficientNet-B0 backbone, combined with Mixup and CutMix augmentations, in addressing the severe class imbalance and capturing modality-specific features such as color, contrast, and texture differences. The errors observed in the test set, accounting for the remaining 0.17%, were analyzed and found to originate from mislabeled samples in the PolypDB dataset. These mislabeled tests, identified through post-hoc analysis of the confusion matrix and manual inspection, indicate inconsistencies in the ground truth annotations, particularly among minority classes (NBI, BLI, FICE, and LCI). Despite these errors, the model's robustness is evident, as it correctly classified the vast majority of samples, even in the presence of noisy labels.

Production Considerations

To ensure the model's deployability in real-world medical environments, we focused on production-ready optimizations, emphasizing lightweight architectures. These choices facilitate efficient inference on resource-constrained devices, such as edge computing setups in clinical settings. Key considerations include containerization for portability and model export to ONNX for cross-framework compatibility and accelerated inference.

Docker

Docker containerization was employed to package the model, dependencies, and runtime environment into a portable, reproducible unit. This approach addresses challenges in medical AI deployment, such as varying hardware (e.g., GPUs in hospitals) and software stacks, ensuring consistent performance across environments.

The Dockerfile is structured as follows:

- **Base Image:** Utilizes NVIDIA's CUDA-enabled PyTorch image for GPU support, falling back to CPU-only for lighter deployments.
- **Dependencies:** Installs essential libraries via `requirements.txt`, including PyTorch, torchvision, ONNX Runtime, and medical imaging tools like OpenCV.
- **Model Loading:** Copies the trained model weights and inference script (`infer.py`) that loads the model, preprocesses colonoscopy images (resize to 224x224, normalization), and outputs modality predictions.

ONNX

For enhanced interoperability and performance, the PyTorch model was exported to ONNX (Open Neural Network Exchange) format, enabling deployment across diverse runtimes like ONNX Runtime, TensorRT, or OpenVINO. This is crucial for medical applications requiring low-latency inference on heterogeneous hardware, speeding up predictions by 2-3x via optimizations.

The export process used PyTorch's `torch.onnx.export` function:

```
import torch
model = torch.load('model.pth')
model.eval()
dummy_input = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, dummy_input, 'model.onnx',
                  export_params=True, opset_version=11,
                  do_constant_folding=True,
                  input_names=['input'], output_names=['output'])
```

Post-export, the ONNX model was validated with `onnx.checker.check_model(model.onnx)` and optimized using ONNX Runtime's graph optimizations. Inference in production leverages ONNX Runtime for CPU/GPU acceleration:

- **Preprocessing:** Consistent with training (e.g., ImageNet normalization).
- **Inference:** `session.run(None, { 'input': input_tensor })` yields softmax probabilities for the five modalities.
Example usage of inference: `python iMIAP_PolyProject_Inference_onnx.py --model efficientnet_model.onnx --input test_images --output full_test_results.csv`
- **Post-processing:** Argmax for classification, with confidence thresholds to flag uncertain predictions.

This ONNX pipeline integrates seamlessly into the Docker container, supporting real-time colonoscopy modality detection in production workflows, with benchmarks showing <50ms inference time on standard hardware.